

Policy Claims Tracker - Application Architecture Cheat Sheet

Student capstone solution overview | React + TypeScript + Express + MongoDB

BIG PICTURE

1. Client / Presentation Layer

React 19 + TypeScript + Vite

Single Page Application (SPA) rendered in the browser.

Routing

/login, /register, /, /claims, /claims/:id, /policies

Pages

DashboardPage, ClaimsPage, ClaimDetailPage, PoliciesPage

Shared components

Navbar + ProtectedRoute; AuthContext stores user/session state.

HTTP integration

Axios /api client adds Bearer JWT and handles HTTP 401.

2. Service / API Layer

Node.js + Express + TypeScript

server.ts configures CORS, JSON parsing, routes and final error middleware.

REST routes

/api/auth, /api/dashboard, /api/policies, /api/claims

Middleware

JWT authentication, express-validator validation, centralized error handling.

Feature logic

CRUD, filtering, pagination, notes, status updates, dashboard aggregation.

Persistence boundary

Mongoose ODM maps TypeScript models to MongoDB documents.

3. Data Layer

MongoDB 7

Database: policy-claims

User

name, email, password hash, role

Policy

policy number, holder, type, premium, status, dates, owner

Claim

claim number, policy ref, incident, amount, status, assignedTo, notes[]

Persistence

Docker volume in Compose; persistent volume claim in Kubernetes.

Mental model: the browser never talks directly to MongoDB. React calls the Express REST API; Express authenticates and validates the request, uses Mongoose to read/write MongoDB, and returns JSON for React to render.

Component Map - What Lives Where?

Use this page to translate requirements into files and responsibilities

COMPONENTS

React Client

- App shell: `src/main.tsx`, `src/App.tsx`
- Authentication: `AuthContext.tsx`, `ProtectedRoute.tsx`, `LoginPage.tsx`, `RegisterPage.tsx`
- Features: `DashboardPage.tsx`, `ClaimsPage.tsx`, `ClaimDetailPage.tsx`, `PoliciesPage.tsx`
- Integration: `api.ts` creates Axios instance, injects token, redirects on 401.

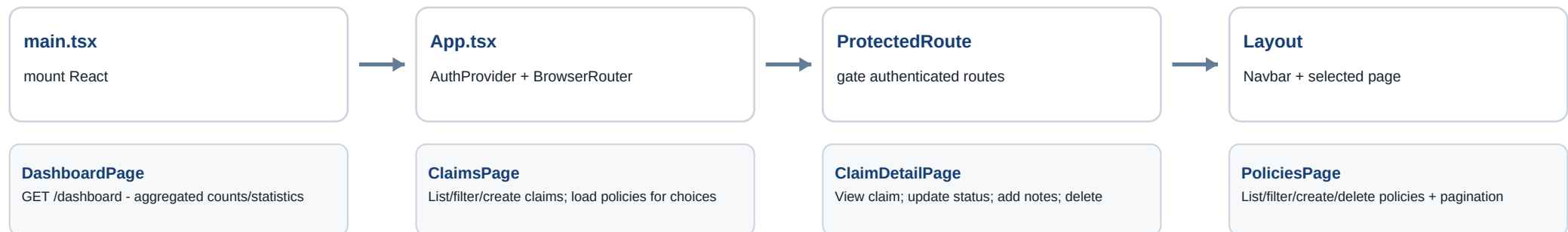
Express API

- Bootstrap: `src/server.ts`, `src/config/db.ts`
- Routes: `auth.ts`, `dashboard.ts`, `policies.ts`, `claims.ts`
- Middleware: `auth.ts`, `validate.ts`, `errorHandler.ts`
- Models: `User.ts`, `Policy.ts`, `Claim.ts`

Infrastructure

- `docker-compose.yml`: client + API + MongoDB
- `docker-compose.prod.yml`: Nginx/SSL production stack
- `k8s/`: namespace, client, API, Mongo, secrets, Kind config
- Testing: Vitest + React Testing Library tests in `client/src/tests`

React Component Relationships

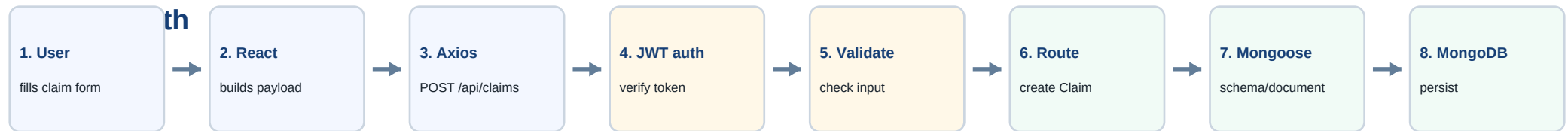


Student build strategy: build one vertical slice at a time - model -> route -> API test -> React page -> integration. Avoid building the entire UI before the API and data model work.

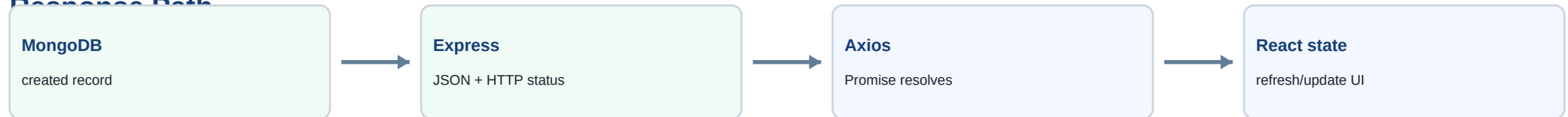
Data Flow - From Click to Database and Back

Example: an authenticated user creates a new claim

DATA FLOW



Response Path



Authentication Flow

- Login calls POST /api/auth/login.
- API loads User with password hash and compares using bcrypt.
- API signs a JWT containing user id; token expires in 7 days.
- AuthContext stores token + user in localStorage.
- Axios adds Authorization: Bearer <JWT> to requests.
- auth.ts verifies token and loads the current user.
- Client clears local session and redirects after HTTP 401.

Validation / Error Flow

- Route validation rules run before business logic.
- Invalid input returns HTTP 400 with details.
- Missing/invalid JWT returns HTTP 401.
- Duplicate values may return HTTP 409.
- Mongoose and unexpected errors converge on errorHandler.ts.
- React catches rejected Axios calls and can show a page-level error.

Architecture idea: authentication, validation, and error handling are middleware concerns. They are reused across features rather than duplicated inside each route.

Domain Model + API Contract

The API contract connects React features to MongoDB documents

DATA + REST

User

- name: string
- email: string (unique)
- password: hash, min 8 chars
- role: adjuster | admin
- createdAt: Date

Policy

- policyNumber: unique
- holderName: string
- type: auto | home | life
- premium: number
- status: active | expired | cancelled
- effectiveDate / expirationDate
- owner: User ObjectId

Claim

- claimNumber: unique
- policy: Policy ObjectId
- incidentDate / amount / description
- status: submitted | under-review | approved | denied | closed
- assignedTo?: User ObjectId
- notes[]: author, text, createdAt

User 1 ---- * Policy 1 ---- * Claim * ---- 0..1 User (assignedTo)

Method	Endpoint	Description	Protected?
POST	/api/auth/register	Create account; return JWT + user	No
POST	/api/auth/login	Validate credentials; return JWT + user	No
GET	/api/auth/me	Current authenticated user	Yes
GET	/api/dashboard	Aggregated dashboard statistics	Yes
GET / POST	/api/policies	List/filter/paginate or create policy	Yes
GET / PUT / DELETE	/api/policies/:id	Read, update, delete one policy	Yes
GET / POST	/api/claims	List/filter/paginate or create claim	Yes
GET	/api/claims/stats	Claim statistics	Yes
GET / PUT / DELETE	/api/claims/:id	Read, update, delete one claim	Yes
POST	/api/claims/:id/notes	Add user-authored claim note	Yes
GET	/api/health	Health check	No

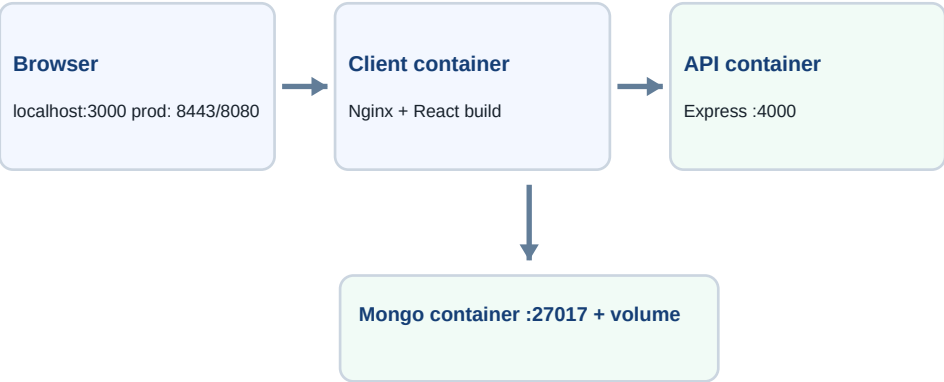
Contract-first thinking: for each screen, identify the endpoint, request shape, response shape, HTTP status/error cases, and required model fields. This keeps frontend and backend work synchronized.

Deployment Architecture + Student Build Roadmap

Same application, different runtime packaging

DEPLOY + BUILD

Docker Compose

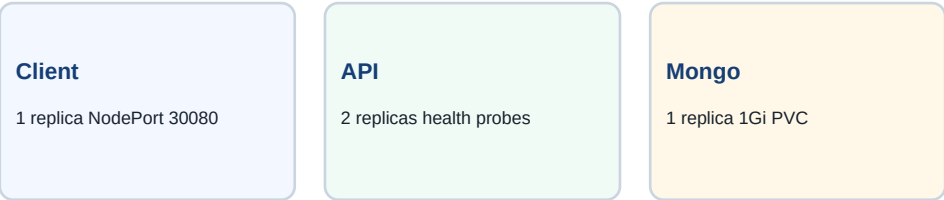


Development Compose exposes client/API/Mongo. Production Compose puts Nginx in front of the built SPA and keeps API/Mongo on the internal container network.

Recommended Student Build Order

#	Build	Evidence it works
1	Project skeleton + environment	Client/API start; Mongo connects
2	Mongoose models + seed	Users, policies, claims in MongoDB
3	Auth + JWT middleware	Register/login; protected route rejects no token
4	Policy CRUD API	CRUD verified with REST client
5	Claim CRUD + notes + filters	Claim lifecycle works through API
6	Dashboard aggregation	Statistics endpoint returns totals
7	React auth + routing	Token persists; ProtectedRoute works
8	React feature pages	UI performs policy/claim operations
9	Errors + validation + tests	400/401/409 handled; tests pass
10	Docker / Kubernetes	Full stack runs; /api/health is green

Kubernetes / Kind



Secrets provide JWT_SECRET and MONGODB_URI to the API. Mongo uses a persistent volume claim; the API has readiness and liveness probes.

Before submission
Environment values documented | README run steps verified | Seed/test data included | No production secrets hard-coded

Functional checklist
Auth works end-to-end | Policy + claim CRUD works | Filters/pagination work | Dashboard reflects current data

Architecture checklist
Client calls API - not DB | Middleware reused across routes | Models enforce data rules | Containers use service names